

Deep Classification Trees (Experimental)

mod classifier

Deep-forest classification estimator built on Keras.

This module provides :class: `~tree_machine.deep_trees.classifier.DFClassifier`, an sklearn-compatible classifier that builds a differentiable forest model using Keras.

The estimator follows the familiar `fit / predict / predict_proba / score` API and supports both built-in and user-provided Keras losses/metrics.

class DFClassifier

Bases: `BaseDeep`, `ClassifierMixin`

A deep-forest classifier with an sklearn-compatible API.

Parameters:

Name	Type	Description	Default
<code>metric</code>	<code>AcceptableMetric</code>	Name of the built-in metric/loss key. Currently only <code>"cross_entropy"</code> is supported.	<code>'cross_entropy'</code>
<code>n_estimators</code>	<code>int</code>	Number of trees/estimators to build.	<code>100</code>
<code>internal_size</code>	<code>int</code>	Internal representation size used by the differentiable tree layers.	<code>12</code>
<code>max_depth</code>	<code>int</code>	Maximum depth of each differentiable tree.	<code>6</code>

<code>feature_fraction</code>	<code>float</code>	Fraction of features sampled per estimator.	<code>1.0</code>
<code>loss</code>	<code>LossLike None</code>	Optional custom Keras loss (callable/instance/class). If omitted (and <code>metrics</code> is also omitted), the built-in loss derived from <code>metric</code> is used.	<code>None</code>
<code>metrics</code>	<code>Sequence[MetricLike] None</code>	Optional sequence of custom Keras metrics (callables/instances/classes).	<code>None</code>
<code>compile_kwargs</code>	<code>dict[str, Any] None</code>	Extra keyword arguments forwarded to <code>Model.compile</code> . If <code>compile_kwargs</code> does not define an optimizer, "adam" is used.	<code>None</code>
<code>decision_l1/decision_l2</code>		L1/L2 regularization strength applied to routing Dense weights.	<i>required</i>
<code>leaf_l1/leaf_l2</code>		L1/L2 regularization strength applied to leaf values.	<i>required</i>
<code>feature_dropout</code>	<code>float</code>	Dropout rate applied to inputs during training.	<code>0.0</code>
<code>routing_dropout</code>	<code>float</code>	Dropout rate applied to routing probabilities during training.	<code>0.0</code>



Notes



- Each call to `:meth: fit` builds and compiles a new Keras model.
- `:meth: score` returns the negative loss value on `(X, y)`.

< > Source code in `src/tree_machine/deep_trees/classifier.py`



98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148

▼ Details

149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175

▼ Details

215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241

▼ Details

281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

▼ Details

▼ Details

▼ Details

98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151

▼ Details

152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178

▼ Details

218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

▼ Details

284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305